

MScFE 642 DEEP LEARNING FOR FINANCE

Group Work Project # 1

[See grading rubric here.](#)

Scenario

Most institutional investors' portfolios have exposure to multiple asset classes and/or geographic areas. This makes a lot of sense, since this way investors can exploit variations in reward-to-risk across asset classes throughout market regimes. Strategic Asset Allocation (SAA) and Tactical Asset Allocation (TAA) are different approaches to multi-asset investment strategies. You can read more about these [in this short article from Vanguard](#).

In this GWP, you will progressively build Deep Learning models that predict short-term market trends across different asset classes. You will start with feedforward and convolutional architectures applied independently to each asset class, then move to recurrent architectures and finally to a unified multi-output model. As the final step, you will test the out-of-sample performance of an asset allocation strategy based on the model's predictions.

Tasks

Step 1

First, you need to gather data on the different asset classes that we are going to consider. To simplify, in this case we will work with 5 different ETFs, each a major representative of their asset class. Here you have their symbols, so that you can work with the *yfinance* library to download the data:

- Equity → SPDR S&P500 ETF. Symbol: "SPY"
- Fixed Income → iShares 20+ Year Treasury Bond ETF. Symbol: "TLT"
- Cash-like → iShares 1-3 Year Treasury Bond ETF. Symbol: "SHY"
- Precious metals (Gold) → SPDR Gold Shares. Symbol: "GLD"
- Crude Oil → Invesco DB Oil Fund. Symbol: "DBO"

Note: *Groups of 2 or 1 students are free to work using only 3 out of the 5 different ETFs (free choice). This will apply throughout the whole assignment.*

- Use the *yfinance* library to download **daily prices** for each of the 5 ETFs. You will use data in the period from January 1st, 2018 to December 30th, 2022 for your **test sample**. We are leaving to your discretion the definition of validation and training sample periods.
- Perform some **exploratory data analysis** on the ETFs. Use visualization tools to describe the dynamics and characteristics of the time series (summary statistics, seasonality, stationarity, ...). Keep it brief and focus on the main characteristics worth highlighting.

The exploratory data analysis should address both individual distributions AND joint distributions, including correlations/covariances.

- Throughout this assignment you may work with the time series in any **stationary representation** you find appropriate (typically daily returns). Justify your choice – for example, by means of an Augmented Dickey-Fuller (ADF) test on the series you ultimately feed into the models.

Step 2

Now you will build your first family of Deep Learning models: a Multi-Layer Perceptron (MLP, or Artificial Neural Network) for each asset class.

- For each of the 5 ETFs, build and train an **MLP** that is designed to predict the **25-day ahead return** of that ETF, using past information of its own time series as inputs (you are free to include whatever lags you consider appropriate).
- Train each model and comment on its **in-sample** predictive performance.
- Test each model **out-of-sample** and comment on its predictive performance, as well as on the comparative results across asset classes.

***Note:** “past information” refers to including several lags of the time series; feel free to include as much or as little information as you wish. You can set the models as regression or classification problems (or both).*

Step 3

Now you are going to compare the performance of the MLPs with Convolutional Neural Network (CNN) architectures. For that, you are going to transform each of the 5 stationary time series and obtain their image representation using Gramian Angular Field (GAF).

- For each of the 5 ETFs, using the **GAF representation** of its (stationary) time series, build and train a **CNN** that is designed to predict the 25-day ahead return of that ETF.
- Evaluate the in-sample and out-of-sample predictive performance of the 5 CNN models, comparing across asset classes and against the MLPs of Step 2.
- As a group, provide arguments on why some models perform better than others – both within the CNN family and in the MLP-vs-CNN comparison – relating your reasoning to the way each architecture processes the data and to the properties of the particular time series.

Step 4

Next, you will create Deep Learning models using recurrent neural networks to predict the 25-day ahead return of each of the 5 different asset classes.

- For each of the 5 ETFs, create a Deep Learning model that uses **LSTM** network architecture (you may combine LSTM layers with regular dense layers) to **predict the 25-day ahead return of each ETF**. Use as inputs the past information of the time series

(feel free to include whatever lags in returns you consider appropriate). You are free to select the model architecture you like.

- Train each of the 5 models. Comment on the in-sample predictive performance of the models, as well as the comparative results across asset classes.
- Test each of the 5 models **out-of-sample**. Comment on the out-of-sample predictive performance of the models, as well as the comparative results across asset classes.
- Compare the LSTM results against the MLP results of Step 2 and the CNN-GAF results of Step 3. Which architecture appears better suited to which asset classes, and why?

Step 5

So far, our models have always had 1 output unit. You will now implement a Deep Learning model with multi-output. If you need more information on multi-output regression, feel free to research online.

- Create a **multi-output model** that blends together all of the information from the 5 LSTM models of Step 4. Specifically, create a model architecture that uses all the inputs from the 5 different previous models together to predict the 25-day ahead return from the 5 different ETFs. In other words, the model will have 5 outputs (one for each 25-day ahead return of an ETF).
- **Train and test** the previous model. Comment on the in- and out-of-sample predictive performance of the model and compare them with the ones from Step 4.
- Discuss whether the multi-output architecture captures information that the single-output LSTMs of Step 4 could not capture (e.g., cross-asset dependencies). Is the same information being used by both architectures?

Groups of 2 or 1 students: You will develop a multi-output model, but it will only contain the output from 3 rather than 5 different ETFs.

Step 6

You will now translate the predictions of your multi-output model into an asset allocation strategy and backtest it.

- Create a **trading strategy that uses the out-of-sample predictions of the multi-output model of Step 5**. You can choose the strategy that you want. For example, you could decide to go long on the 2 (or 3) asset classes the model predicts will do best and go short on the 2 (or 3) asset classes the model expects to do worse. **The strategy needs to be rebalanced every 25 days and use information from the predictions of at least 4 of the 5 asset classes.**
- **Backtest** the previous strategy in the test period. Comment on the results. Compare the results versus the **Buy-and-hold return of an Equally Weighted portfolio** of the 5 asset classes.

- Optionally, build an analogous strategy using the predictions of the 5 single-output LSTMs of Step 4 and compare its backtest against the multi-output strategy. This helps you isolate the value added by the multi-output architecture from the value added by the LSTM itself.

Groups of 2 or 1 students: *The strategy must use information from the predictions of at least 2 of the 3 asset classes.*

Step 7

As a group, discuss the different results obtained throughout the assignment. Specifically:

- What are the comparative predictability properties of MLP, CNN-GAF and LSTM architectures across the different asset classes? Can you tie the differences to how each architecture processes information and to the statistical properties of each asset class?
- What are the predictability implications of the multi-output versus the single-output architectures? Are they capturing the same information?
- How does the backtesting performance of the multi-output strategy compare to the Buy-and-Hold Equally Weighted benchmark? What does that imply about the practical value of multi-asset prediction in this setup?

Step 8

As a group, please submit all files that reproduce the assignment. This will include the Python file for all the steps as well as work together on a report that contains the answers to all the questions above (except for the code) in a clear and organized manner (think about how you would present this to your boss).

Submission Requirements and Format

One team member submits the following on behalf of the entire group:

- **1 PDF document** containing ONLY the answers to the questions, EXCLUDING code.
- Use the available Report Template and fill out the required information in the first page.
- **1 zipped folder** that contains:
 - Jupyter notebook that is executable
 - html of both the Jupyter notebook containing all code, results, and graphs

Use Google Colab or GitHub to collaborate in completing the executable Python program.

NOTE: The PDF must be uploaded **separately** from the zipped folder that includes any other types of files. This allows Turnitin to generate a similarity report.

Rubric

Your instructor will evaluate your group submission for GWP1 using the following rubric:

Quantitative Analysis (Open-Ended Questions)	Technical and Non-Technical Reports	Writing and Formatting
60 Points	40 Points	20 Points
The group is able to apply results, formulas, and their knowledge of theory to real-life finance scenarios by doing the following: • Providing all the necessary information to support their arguments. • Presenting arguments that reflect group discussion and research. • Using authoritative references to support a position and provide updated information. • Concluding with practical takeaways for more insightful financial decision-making.	Technical reports contain 3 parts: 1) code for each question (be sure to explicitly state the question number), 2) the corresponding output of that code, and 3) <u>interpretations and/or recommended courses of action that reasonably follow from those results.</u> Note: Technical reports will include the technicalities of models, such as names, methods of estimation, parameter values, etc., and exclude generalities about the work done. It should NOT include names of Python code that were used.	A submission that looks professional should: • Include the axes, labels, and scales in graphs. • Be free of significant grammatical errors or typos. • Be an organized, well-structured, and easy-to-read document. • Include proper citations and a bibliography in MLA format.
	Non-technical reports contain 3 parts: 1) clear explanation of results; 2) the recommended course of action that follows; and 3) the identification of factors that impact each portfolio. Note: AVOID all references to model names, algorithms, and unnecessary details. Instead, focus on the investment decision.	

Revised: May 15, 2026